

Project Blueprint: Hybrid E-Commerce Arbitrage Swarm

1. Executive Summary

This project is a hybrid multi-agent system designed to identify, evaluate, and output profitable arbitrage opportunities in secondary e-commerce markets. By utilizing a hybrid architecture—relying heavily on deterministic code for heavy lifting and a swarm of lightweight, specialized LLM agents for reasoning tasks—the system achieves high efficiency and minimal token costs.

2. Pros and Cons

Pros

- **True Multi-Agent Architecture:** Satisfies university requirements by demonstrating complex agent-to-agent communication and task delegation using frameworks like AutoGen or LangGraph.
- **High Technical "Wow" Factor:** Demonstrates mastery of data pipelines, API integration, and cost-optimized AI usage.
- **Real-World Applicability:** Solves an actual financial/e-commerce problem rather than just summarizing text.
- **Extremely Cost-Efficient:** Bypasses the massive API token costs usually associated with naive multi-agent systems.

Cons

- **Scraper Fragility:** E-commerce sites update their UI frequently. If a site changes its HTML structure, your scraping code will break and need adjustments.
- **Anti-Bot Protections:** Sites like Shopee use Cloudflare to block scrapers. You will need to use stealth techniques.
- **Latency:** Scraping multiple pages takes time. The system will need to run asynchronously to be fast.

3. How It Works (The Multi-Agent Swarm Protocol)

The system operates as an automated pipeline where standard code filters data, and then hands it off to a **Swarm of 3 distinct AI Agents** that collaborate to make a final decision.

Step-by-Step Workflow

1. **Data Ingestion & Hard-Filtering (Deterministic - 0 Tokens):** Python scrapers extract live listings. Standard code instantly drops items that are too expensive, out of stock, or

from low-rated sellers.

2. **The Multi-Agent Handoff (The Swarm is Activated):** The filtered listings are passed to the Agent Orchestrator.
3. **Agent 1: The Context Analyst (LLM Text Agent):**
 - *Task:* Reads the messy seller description.
 - *Output:* Extracts the true condition (e.g., "Used", "Mint"), checks for missing parts, and flags seller red flags (e.g., "Box is slightly dented"). Passes findings to Agent 3.
4. **Agent 2: The Vision Authenticator (Multi-Modal Agent):**
 - *Task:* Receives the listing images. Compares the images against known authentic benchmarks (looking for fake holographic stickers, wrong colors).
 - *Output:* A confidence score (0-100%) on authenticity. Passes findings to Agent 3.
5. **Agent 3: The Lead Strategist (Decision Agent):**
 - *Task:* Takes the condition from Agent 1, the authenticity score from Agent 2, and the math logic (shipping + platform fees).
 - *Output:* Makes the final decision. If profitable, it generates a custom negotiation message (e.g., "Since the box is dented, will you take RM 50?") and pushes the deal to your dashboard.

4. System Architecture Diagram

Below is the visual workflow from the user's initial prompt to the final output, showing exactly how tasks are split among the agents.

graph TD

A([User Prompts Dashboard: e.g., "Find Labubu Macaron under RM 60"]) --> B[Python Web Scraper]

subgraph "Phase 1: Deterministic Code (0 Tokens)"

B --> |Raw HTML/Data| C{Hard-Filter Logic}

C --> |Overpriced/Low Rating| D[Discard Junk]

C --> |Cleaned JSON Listings| E[Agent Orchestrator]

end

subgraph "Phase 2: The Multi-Agent Swarm"

E --> |Sends Listing Text| F(Agent 1: Context Analyst)

E --> |Sends Listing Images| G(Agent 2: Vision Authenticator)

F --> |Extracts Condition & Flaws| H(Agent 3: Lead Strategist)

G --> |Image Authenticity Score| H

end

subgraph "Phase 3: Execution"

H --> |{Is Margin > 20% ?}| I

I --> |No| J[Pass on Deal]

```
I -->|Yes| K([Generate Negotiation Message])
L([Push Alert to Telegram & Web App])
K --> L
end
```

```
style A fill:#4CAF50,stroke:#388E3C,stroke-width:2px,color:#fff
style E fill:#FF9800,stroke:#F57C00,stroke-width:2px,color:#fff
style F fill:#2196F3,stroke:#1976D2,stroke-width:2px,color:#fff
style G fill:#2196F3,stroke:#1976D2,stroke-width:2px,color:#fff
style H fill:#9C27B0,stroke:#7B1FA2,stroke-width:2px,color:#fff
style L fill:#F44336,stroke:#D32F2F,stroke-width:2px,color:#fff
```

5. Tech Stack & Tool Requirements

- **Multi-Agent Framework:** LangGraph (great for strict control) or Microsoft AutoGen (great for letting agents chat with each other naturally).
- **Language:** Python 3.10+
- **Web Scraping:** Playwright or BeautifulSoup.
- **AI Models:** Gemini 1.5 Flash or OpenAI GPT-4o-mini for incredibly fast, cheap reasoning.
- **Data Validation:** Pydantic (Forces the agents to only output strict JSON, preventing rambling text).
- **Frontend (For Demo):** Streamlit or a basic FastAPI backend.

6. Development Plan (Start to Finish)

Phase 1: The Foundation (Data Collection)

- *Goal:* Build the deterministic scrapers. Choose one target marketplace (e.g., Carousell Malaysia). Write a Python script to search for a specific item, extract the top 20 listings, and save the data to a JSON file.

Phase 2: The Filter Pipeline (Logic)

- *Goal:* Clean the data without AI. Write Python functions to iterate through your JSON data and remove any junk items.

Phase 3: Building the Agent Swarm (The Core Project)

- *Goal:* Setup the Multi-Agent framework (AutoGen/LangChain).
- *Tasks:* Define the 3 distinct agents (Analyst, Authenticator, Strategist). Write the system prompts for each. Establish the communication protocol so Agent 1's output feeds directly into Agent 2, and so on.

Phase 4: The Margin Calculator & Execution

- *Goal:* Finalize the business logic. Write the math function to calculate final profit. Set up a webhook (Discord/Telegram) for Agent 3 to send its final alerts.

Phase 5: The Dashboard (University Demo Prep)

- *Goal:* Make it look cool for the lecturers. Build a simple Streamlit dashboard. Add a "Start Swarm" button that shows a live-updating table of the agents "talking" to each other and making decisions in real-time.